Table of Contents

目的、大致效果预览 2

Github 需要用到的板块 3

如何规范提交代码（任务发布到完成的全流程） 3

步骤速通 3

详细步骤 3

1. 在 Issues 中发布一个 new issue 3
2. 开始工作，将 project 中的 issue 移到 in progress 5
3. 克隆到本地，切换分支 5
4. 在分支中开发 5
5. 开发完，本地分支合并到 github 分支 5
6. 提交 pull request 6
7. 让其他人审核并通过 (审核人视角) 7
8. 将 project 中的 issue 移到 done 8
9. 删除无用的分支 8

其他问题 9

1. 分支怎么命名（重要） 9
2. 多人开发时，合并代码容易出现的问题（重要） 9
3. Issues 发布的任务的体量多少合适（重要） 10
4. Issues 的 checklist 功能在哪 10
5. 如果任务（Issue）跨周怎么办 10
6. PR 建错了怎么删 10
7. 分支可以共用吗 10
8. 分支什么时候删掉 10

其他注意事项 10

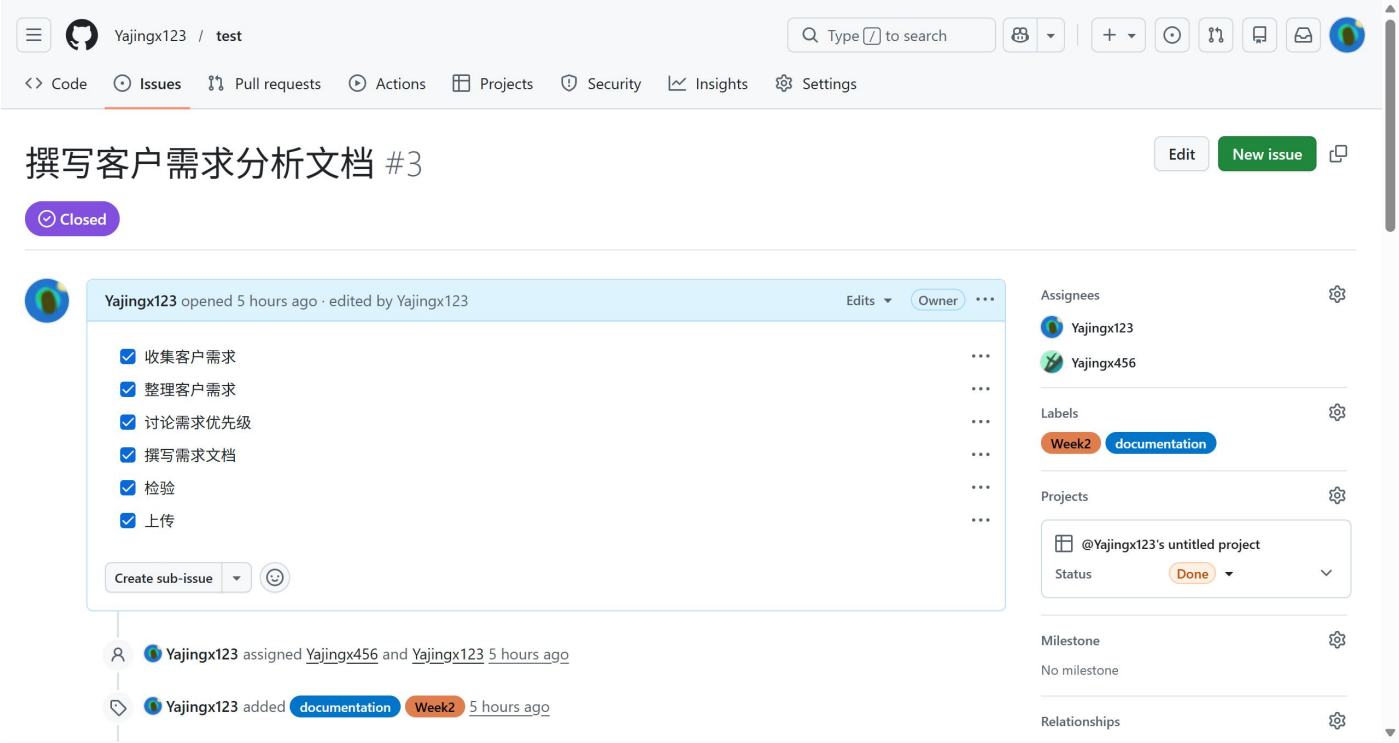
如何对指南中提到的功能进行练习 10

目的、大致效果预览

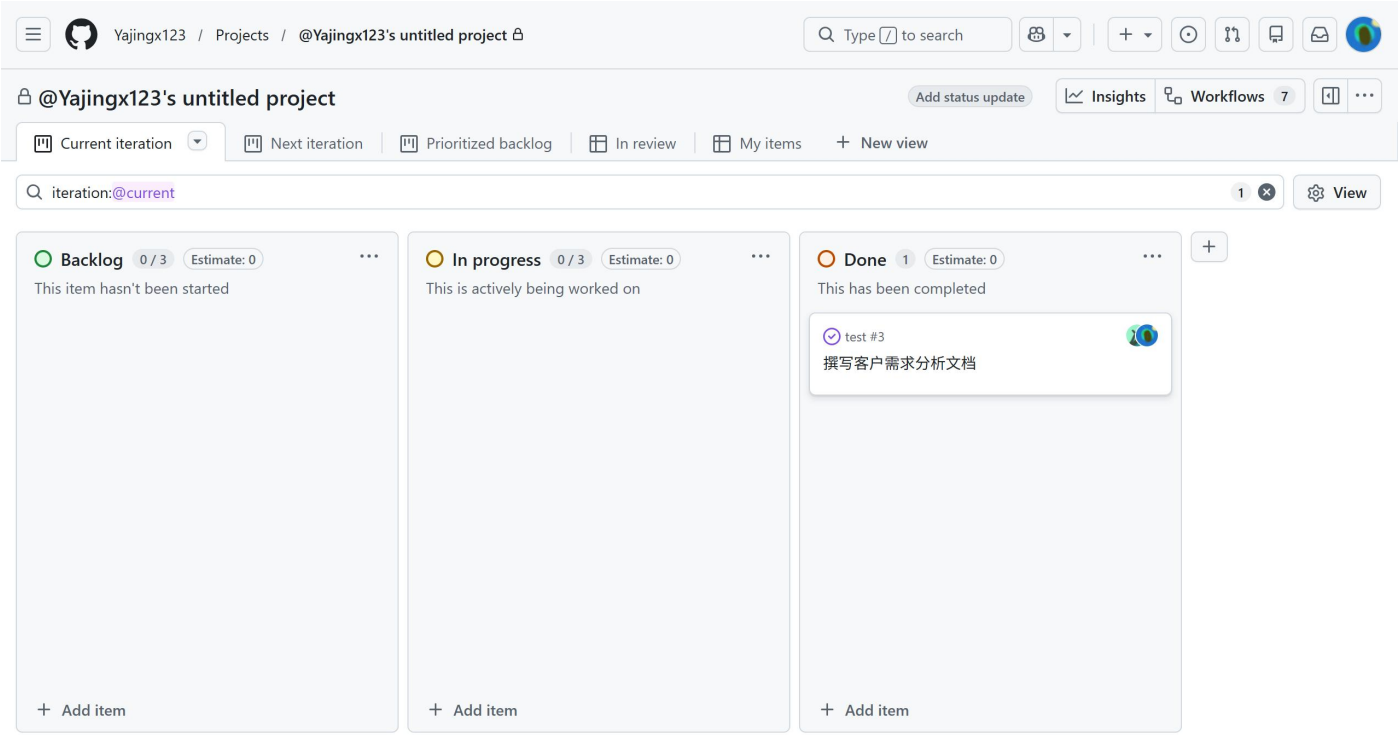
目的：任务可视化，了解所有人当前任务内容及进度

大致效果预览：

1. 单个任务可视化的效果：



2. 任务进度效果（这里只放了一个）：



Github 需要用到的板块

只需要用到这四个

Code：存放代码，不同的功能在不同分支上开发，开发完及时删分支

Issues：发布任务，分配给不同的组员，能在 project 里查看进度

Pull requests（PR）：在分支上开发完，发布一个'请求（PR）'来合并到主分支，必须由一人审核

Projects：追踪 issue 进度，能知道所有人都在做什么、每周做了什么

如何规范提交代码（任务发布到完成的全流程）

步骤速通

1. 在 Issues 中发布一个 new issue
2. 开始工作，将 project 中的 issue 移到 in progress
3. 克隆到本地，切换分支
4. 在分支中开发
5. 开发完，本地分支合并到 github 分支
6. 提交 pull request
7. 让其他人审核并通过
8. 将 project 中的 issue 移到 done
9. 删除无用的分支

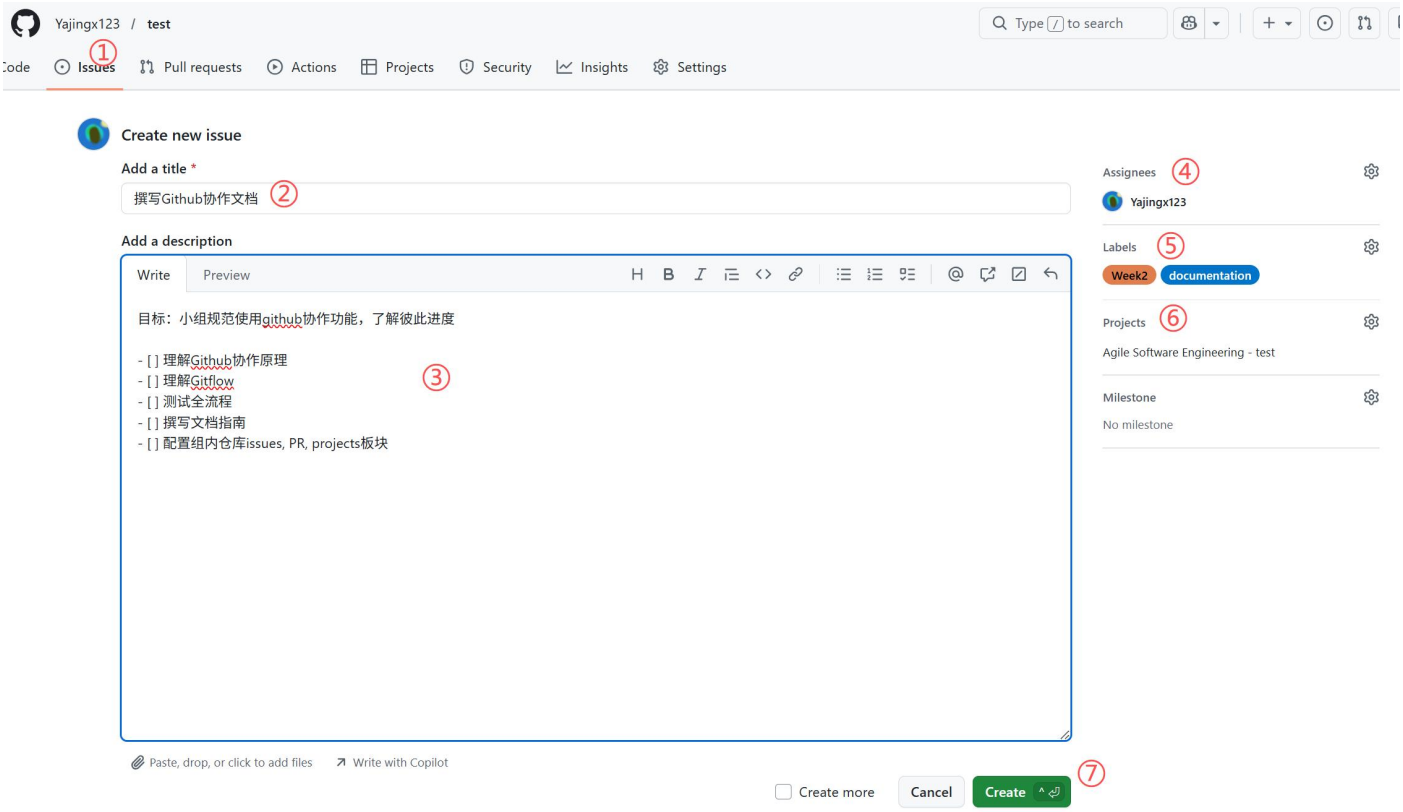
详细步骤

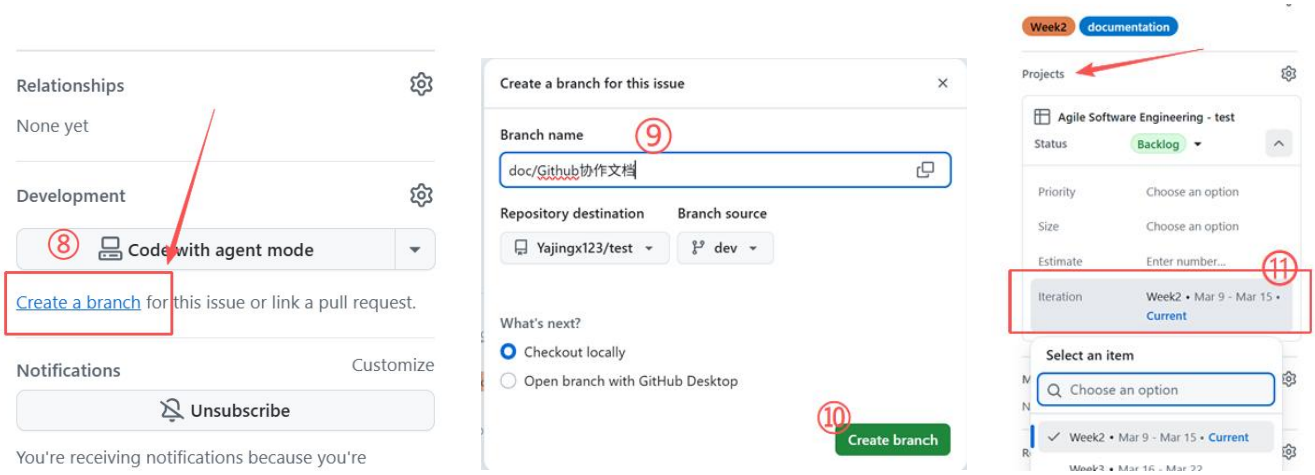
1. 在 Issues 中发布一个 new issue

- Issues -> new issue
- 完善 title, description, assignee, lables, projects 信息
- Create
- 在发布后的任务界面选择 create a branch (上传文档或图片也需要建，因为 PR 的强制审核功能不让你对 dev 分支随意添加文件)
- 在发布后的任务界面选择选择 project-iteration

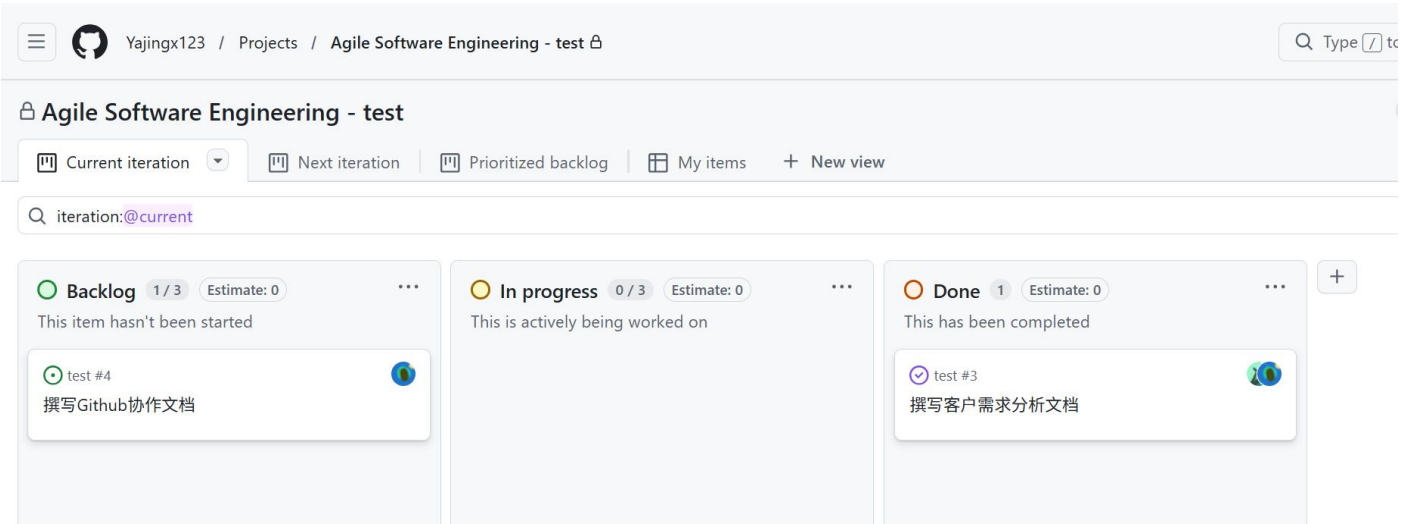
注意：

- 任务的体量大小大概是一个人 2 小时 ~ 一天就能完成的量
- assignee 只能有一人，代表该任务负责人，若有其他参与者可以放 description
- lables 需至少两个，哪一周+任务类型
- 分支命名规则参见底下【其他问题】



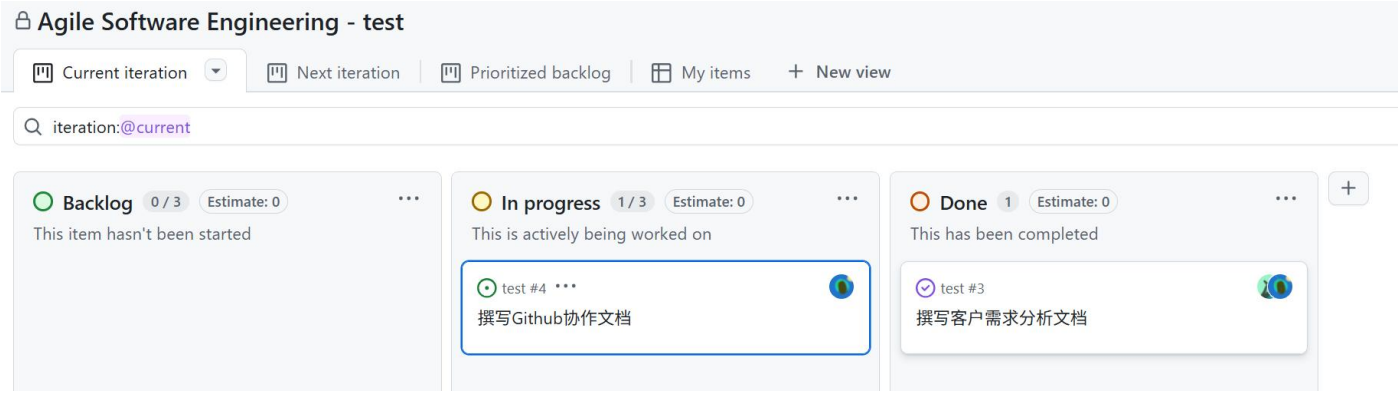


此时 project 的 backlog 中就已有该 issue 了:

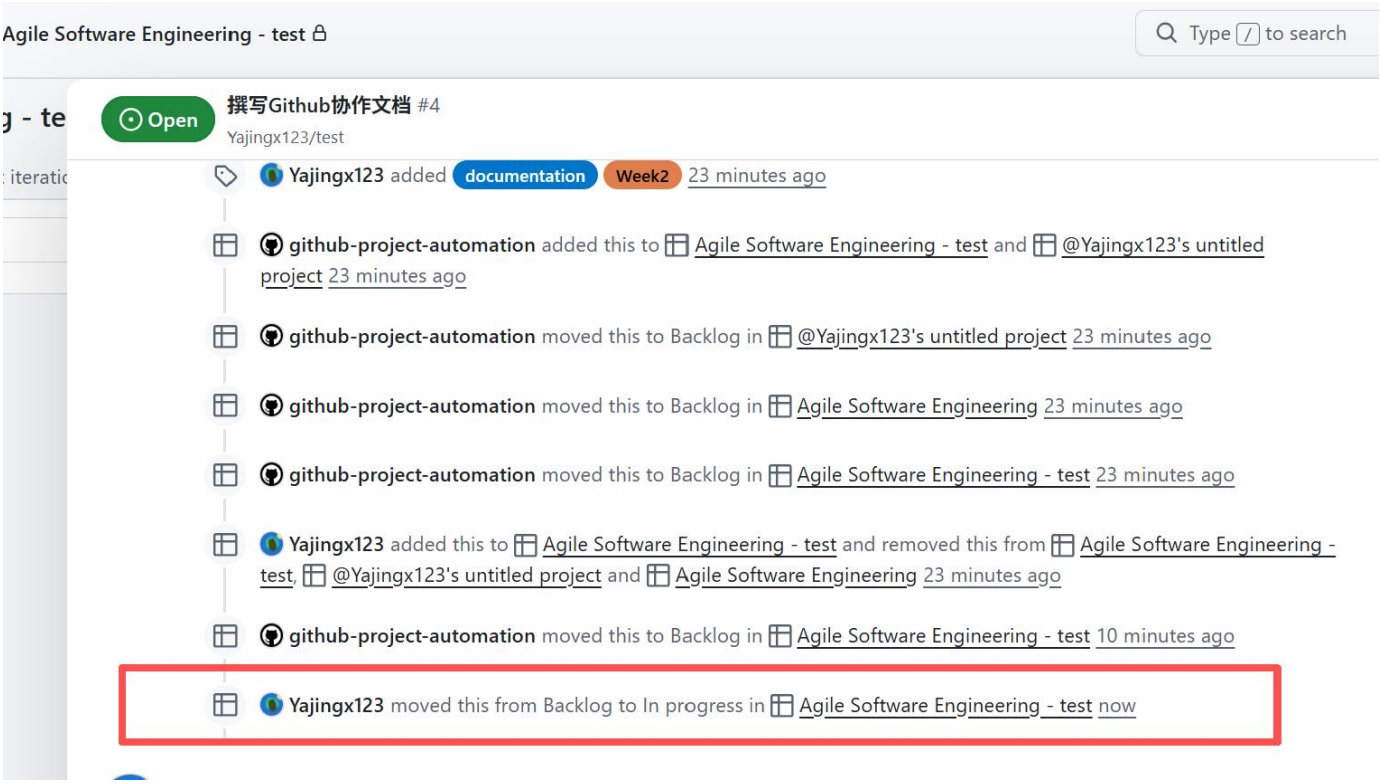


2. 开始工作，将 project 中的 issue 移到 in progress

手动拖动就行



注意：不要频繁拖来拖去地，每次一次拖动都会产生记录，点开该 issue 就能看到



3. 克隆到本地，切换分支

- Git clone …
- Git checkout -b feature/xxx origin/f

4. 在分支中开发

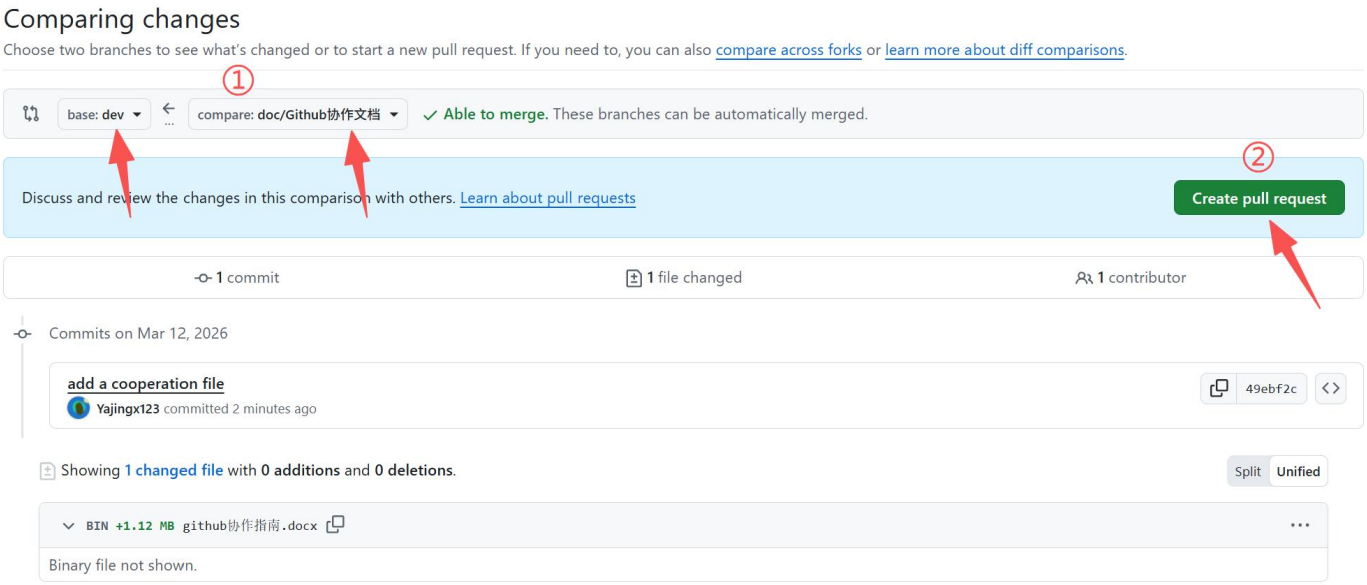
5. 开发完，本地分支合并到 github 分支

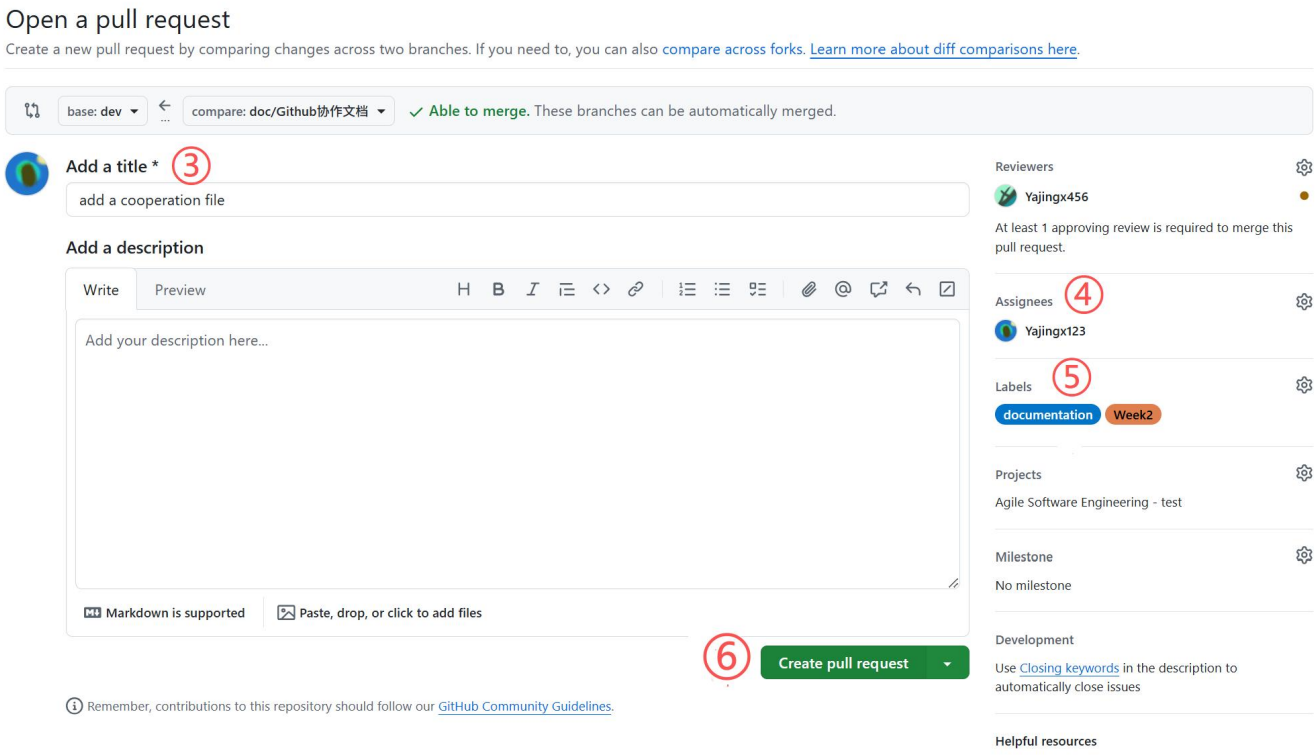
- Git add …
- Git commit -m "…"
- Git push -u origin doc/Github 协作文档

6. 提交 pull request

- Pull requests 界面-> new pull request
- 选择合并的分支(= 将你的修改同步到主开发文件夹中)
- Create pull request
- 必填：title, assignees, labels
- 选填：reviewers (选了就只能由你指定的人审核，不选其他人也能审核)
- Create pull request

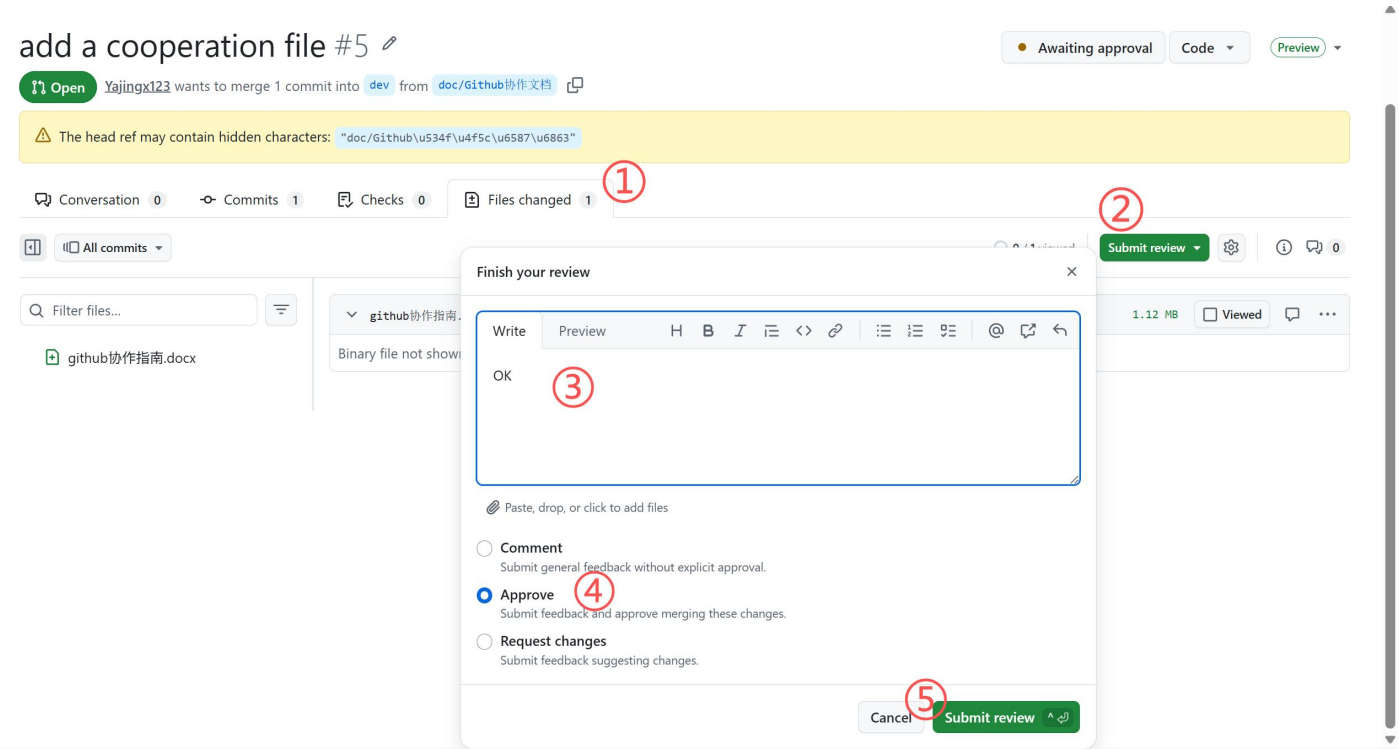
PS： assignees 一般都是发起 PR 的人，但也可以不是你自己，assignees 代表该 PR 的负责人，如果 PR 出现问题须由 ta 来解决（比如代码合并要解决的冲突）

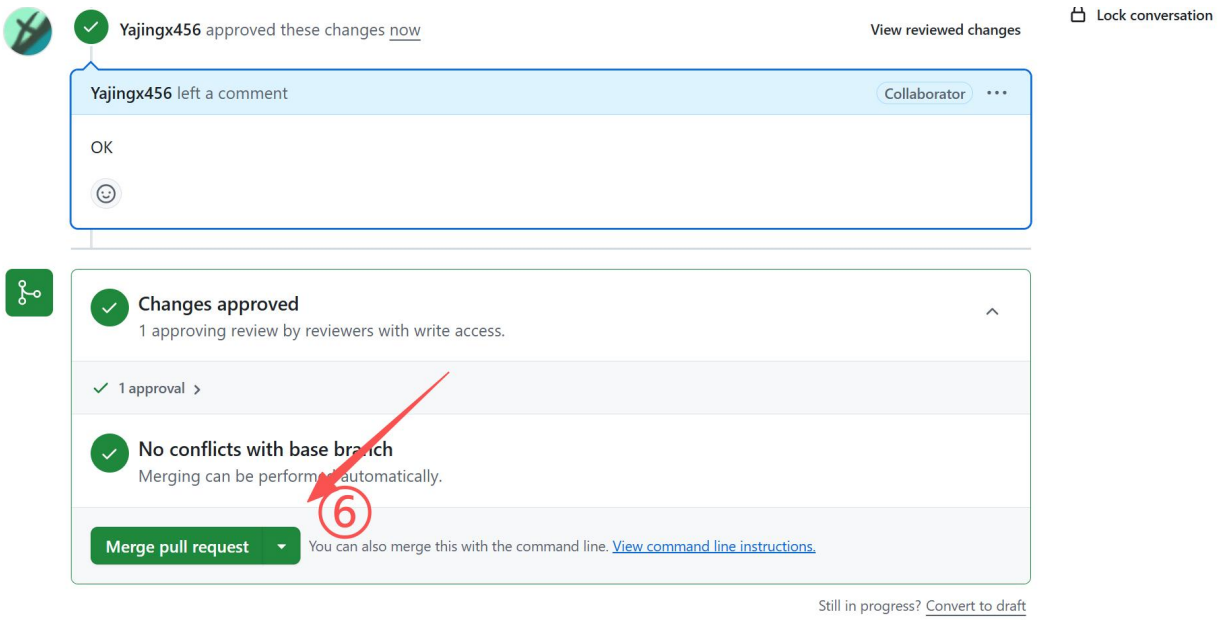




7. 让其他人审核并通过 (审核人视角)

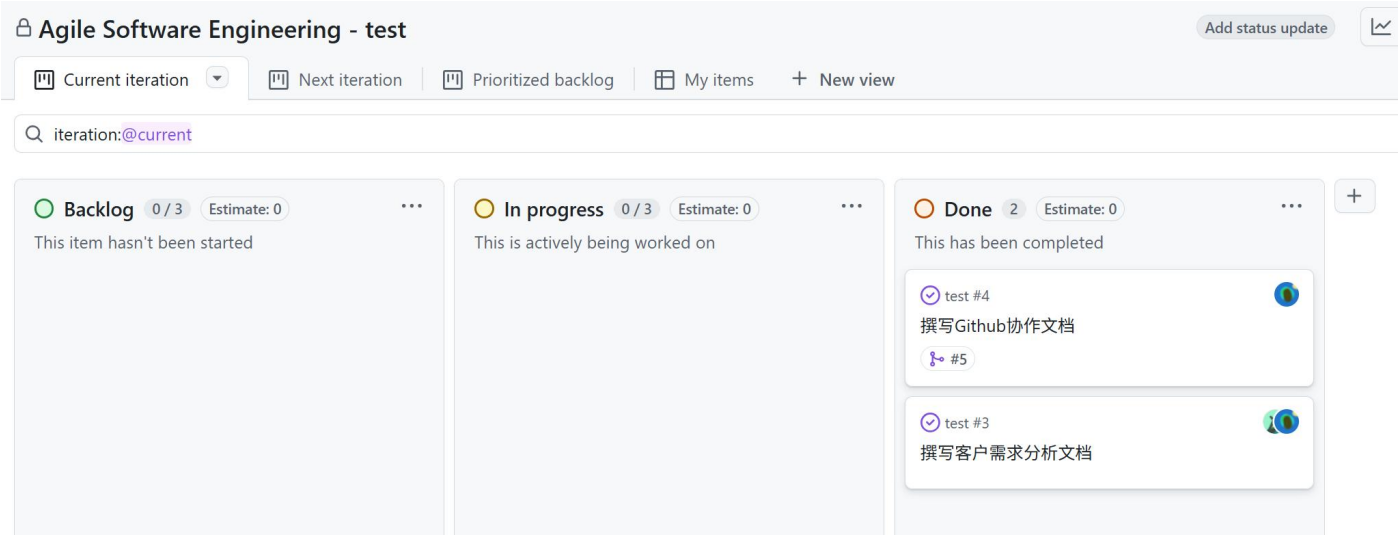
a. 点进你要审核的 PR
b. 选择 Files changed -> submit review
c. 填写 review
d. 选 Approve
e. Submit review
f. 现在就可以 merge pull request 了





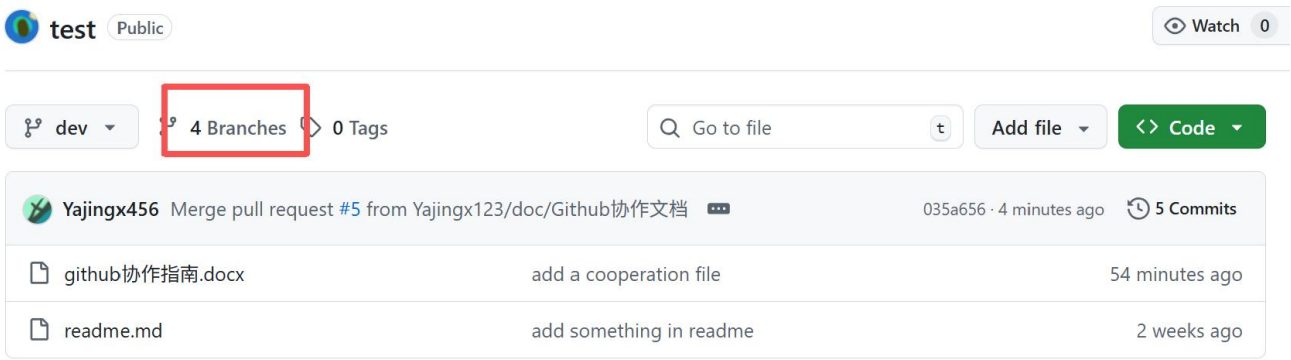
8. 将 project 中的 issue 移到 done

一般如果前面的步骤都完成了，PR 审核完 Github 就会自动把 issue 放入 done，issue 也会自动关闭，如果没有的话，可以手动把 issue 拖到 done，或检查前面步骤，看 issue 的信息是不是有缺漏并补全



9. 删除无用的分支

点进去选择分支并删掉



其他问题

1. 分支怎么命名（重要）

四种分支

分支名	作用	例子
main	最终稳定版，从 dev 合并过来	（固定的，勿动）
dev	日常开发的集成分支 所有人功能最终合并到这里	（固定的，勿动）
feature/xxx	做新功能用 每个人自己的任务 从 dev 切出来	feature/login-page feature/user-db
bugfix/xxx	用来修复已经合并到 dev/main 的 bug 从 dev 切出来	bugfix/login-error
doc/xxx	放文档用的 即用即删 也需要创建分支因为 PR 有强制审核	doc/client-requirements

2. 多人开发时，合并代码容易出现的问题（重要）

【方法来自豆包，实际情况请灵活运对】

情况一：不同的人的 feature 都要合并要一起，如何解决代码冲突

方法：

- **必须先拉 dev 最新代码**：多人并行开发时，dev 会不断更新，提前拉取能避免大面积冲突
- **冲突只在本地解决**：绝对不要直接在 dev 分支改代码，所有冲突在自己的 feature 分支解决
- **一个 feature 只合一次**：做完一个功能合一次，不要攒多个功能一次性合
- **合并后及时删 feature 分支**：避免远程分支泛滥，7 人团队很容易分支混乱

情况二：我在 dev 分支上切出自己的 feature 分支，在开发过程中 dev 被其他组员更新了怎么办

方法：

后提交 PR 到 dev 的人，必须解决自己的 feature 分支与最新 dev 分支（已包含前一个人的代码）之间的所有冲突，包括代码分歧较大的情况

核心是 “先拉最新 dev 到自己的 feature 分支，沟通确认逻辑后手动调整代码”，所以最好：

- **高频同步**：每天至少同步 1 次 dev 的最新代码（尤其是多人改同一模块时
- **模块拆分清晰：**尽量让不同开发者负责不同文件 / 模块（比如 A 做登录页面、B 做注册接口），从根源减少冲突
- **冲突早解决**：发现 dev 更新后立刻同步，不要攒到功能快做完才处理，否则冲突会更多
- **及时沟通：**如果要改核心文件（比如公共配置、通用函数），先在团队群里说一声，避免多人同时改
- **小步提交：**自己的 feature 分支里，完成小功能就提交一次（比如完成登录按钮样式），不要攒大量代码一次性提交

3. Issues 发布的任务的体量多少合适（重要）

别太小（琐碎、浪费管理成本，如修改按钮颜色）

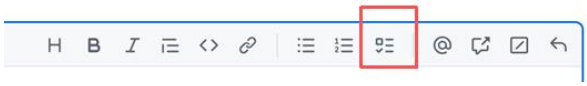
别太大（无法跟踪、容易跨周，如完成听力模块）

最好：一个人 0.5~2 天就能完成（例：设计登录界面 UI/ 配置数据库连接/ 修复登录密码验证 bug）

如果要 2 天：在 description 里面建立 checklist，使之能看见小步骤

如果要>3 天：除了文档工作以外都不建议

4. Issues 的 checklist 功能在哪



5. 如果任务（Issue）跨周怎么办

三种办法：

1. 把该 issue 信息右侧的 project->iteration 选项改成下周
2. 把 issue 拆成两个
3. 直接关闭当前 issue，写上“未完成，移到下周”，再开一个一摸一样的

6. PR 建错了怎么删

无法完全删除，只能“废弃”：PR 界面拉到最底下选 close pull request

7. 分支可以共用吗

NO，每个人开自己的分支，做好自己的任务

8. 分支什么时候删掉

PR 完就删，保持分支干净整洁

其他注意事项

1. Issue 信息要完整，才能自动关联 project 和 branch
2. 分支及时删掉

如何对指南中提到的功能进行练习

三种办法

1. 自己在 Deadline-Disco 仓库中谨慎实操
2. 自己开一个练习用的仓库
3. 进入我用来练习的仓库，随意练习（找我把你加进去，里面有配置好 PR 审核规则，有跟 Deadlind-Disco 仓库一样的 project 模板）